

PROJETO DE DESENVOLVIMENTO DE SISTEMA CORPORATIVO

Profº Dennys Cavalcanti Carvalho



A close-up photograph of a person's hands using a purple marker to draw on a whiteboard. The background is blurred, showing some office equipment and lights. The word 'Ementa' is overlaid in white text on the left side of the image.

Ementa

- Arquiteturas de aplicações corporativas. Programação baseada em componentes.
- Utilização de uma plataforma de programação para o desenvolvimento de aplicações corporativas.
- APIs de persistência de dados.

Objetivo

O objetivo desta cadeira é projetar e desenvolver sistemas corporativos completos, desde a concepção da ideia até a implementação e publicação do sistema.

Projeto para Conclusão de Curso

O Trabalho de Conclusão do Curso não é parte integrante da matriz Curricular do Curso, sendo a conclusão do curso a realização de Projetos de Desenvolvimento de Softwares. Está previsto neste PPC a realização de 2 (dois) projetos: Projeto de Desenvolvimento de Sistemas Web (Módulo IV) e Projeto de Desenvolvimento de Sistema Corporativo (Módulo V) como requisito para a conclusão do Curso. O desenvolvimento de software terá como objetivo:

- a) o desenvolvimento de um software, em que o estudante concluinte precise colocar em prática, de forma integrada, os conhecimentos trabalhados ao longo do curso
- b) está relacionado à construção do software, tanto em aspectos de codificação, quanto de documentação, análise e projeto, testes, dentre outros possíveis; e
- c) requer a escrita de um relatório técnico

A avaliação será realizada pela a caracterização de um trabalho escrito, que será desenvolvido na forma de relatório técnico. Os produtos de trabalho como softwares, rotinas de código e protótipos desenvolvidos também podem integrar a avaliação, assim como a apresentação do trabalho, como formas de comprovar, exemplificar e sustentar o conteúdo do documento escrito.

Pré-requisito

- Conhecimento prévio de HTML, CSS e JavaScript.
- Familiaridade com pelo menos um framework front-end (React, Angular ou Vue).
- Conhecimento básico de desenvolvimento back-end (Node.js, SpringBoot, Django).
- Noções básicas de banco de dados (SQL, MySQL, PostgreSQL, mongoDB, Redis, Firebase).
- Familiaridade com versionamento de código (Git).
- Adoção e especificação de Design Patterns.
- Apresentar coberturas de **testes funcionais** (unitários, integração e interface), **testes de qualidade** (desempenho, carga e responsividade) e de **segurança** (https, permissão e integridade de dados)
- Adoção de **Domain-Driven Design (DDD)**

- Explorar documentações oficiais dos frameworks escolhidos.
- Realizar exercícios práticos com APIs RESTful.
- Estudar conceitos de segurança web e boas práticas de autenticação e autorização.
- Dedicar tempo para aprender testes automatizados (Jest, JUnit).
- Familiarizar-se com plataformas de deploy (Heroku, AWS, Vercel).

Critérios da proposta de projeto.

- O projeto deve contemplar uma aplicação web corporativa completa com Front-End e Back-End integrado e com validação de dados.
- Deve permitir uso de pelo menos **um banco relacional** e **um banco noSQL**.
- A aplicação precisa incluir autenticação e autorização (login, cadastro, gerenciamento e Auditoria Rígida de usuários).
- Implementação de CRUD (Create, Read, Update, Delete).
- Aplicação obrigatória de testes automatizados (unitários e integração).
- Documentação técnica detalhada e manual do usuário completos.
- Deploy funcional em um ambiente de homologação.
- Interface responsiva, validada em diferentes dispositivos (desktop, tablet, mobile).
- Aplicação de boas práticas de segurança.

Critérios de aceitação do projeto (corporativo)

- **4 módulos obrigatórios:** gestão de identidade (IAM), core business (domínio), comunicação e eventos (messengeria), módulo de inteligência de dados (dashboard executiva com indicadores)
- **Implementação de health checks via endpoint e estratégias de circuit breaker** para evitar que uma falha em um serviço derrube o sistema inteiro.
- **Auditabilidade:** Todo registro importante deve ter um histórico (Quem criou, quando mudou, qual era o valor antigo)
- **Uso de protocolos OAuth2 e OpenID Connect** com provedores de identidade.
- Uso de brokers Kafka, RabbitMQ para processamento assíncrono.
- **Interoperabilidade:** Capacidade de consumir e expor serviços para sistemas legados ou parceiros externos via protocolos padronizados.

Critérios para Avaliação

- Qualidade Técnica (funcionalidade, estabilidade, segurança).
- Complexidade e Escopo (múltiplos módulos, relações no banco, recursos extras).
- UX/UI (design, responsividade, usabilidade).
- Documentação (escopo, requisitos, diagramas).
- Apresentação (clareza, objetividade, domínio do conteúdo).
- Diferenciais (inovações e recursos não obrigatórios).

Previsão de datas (podem mudar devido a feriados ou outras atividades)

Ciclo 1: Engenharia de Domínio e Modelagem

12/02 - Aula 01: Alinhamento e Concepção

18:40 - 20:20: Apresentação da disciplina, critérios de avaliação e formação de grupos.

20:40 - 22:00: Workshop de Ideação: Definição do tema do sistema corporativo e identificação dos primeiros **Bounded Contexts** (Contextos Delimitados).

19/02 - Aula 02: Engenharia de Requisitos Corporativos

18:40 - 22:00: Redação do **DDE** (Documento de Definição de Escopo)

26/02 - Aula 03: Entrega 01 + Início de DDD

18:40 - 19:30: **ENTREGA 1**: Recebimento do DDE e Diagrama de Casos de Uso.

19:30 - 22:00: **DDD (Domain-Driven Design)**: Identificação de Entidades, Objetos de Valor e a criação da **Linguagem Ubíqua**.

Previsão de datas (podem mudar devido a feriados ou outras atividades)

05/03 - Aula 04: Modelagem Tática

18:40 - 22:00: Desenho do **Diagrama de Classes** focado no Domínio (sem preocupação com banco de dados ainda).

12/03 - Aula 05: Arquitetura (conversão pra hexagonal)

18:40 – 22:00: Refatoração arquitetural e Mapeamento de quais serviços externos serão "Adaptadores".

19/03 - Aula 06: Entrega 02

18:40 - 22:00: ENTREGA 2: Diagrama de Classes Final e Diagrama de Sequência de fluxos críticos.

26/03 - Aula 07: Persistência e Dados

18:40 - 22:20: Modelagem de dados.

Conceitos de ACID e Row Level Security (RLS).

Criação dos scripts SQL e planejamento de migrações/conversões de banco.

Previsão de datas (podem mudar devido a feriados ou outras atividades)

Ciclo 2: Implementação e Autenticação

02/04 - Aula 08: Setup Tecnológico e Docker

18:40 - 20:20: Configuração de ambiente: **Vite**, **TypeScript** e **Docker**.

20:20 - 22:00: Criação do repositório Git e estrutura de pastas seguindo a Arquitetura Hexagonal.

09/04 - Aula 09: Identidade e Acesso (IAM)

18:40 - 20:20: Integração com Provedores de Identidade (Supabase Auth/Clerk/Keycloak).

20:20 - 22:00: Implementação de Login, Cadastro e Recuperação de Senha.

16/04 - Aula 10: Entrega 03 (MVP)

18:40 - 22:00: **ENTREGA 3 (Apresentação em Lab)**: Sistema realizando CRUD básico com autenticação funcional e segurança de tipos (TypeScript).

23/04 - Aula 11: Governança e Permissões (RBAC)

18:40 - 20:20: Aula sobre controle de acesso baseado em funções

20:20 - 22:00: Implementação de permissões granulares no backend e no frontend.

30/04 - Aula 12: Interfaces Corporativas e Dashboards

18:40 - 20:20: UX para sistemas complexos: Dashboards e visualização de dados com **Recharts**.

20:40 - 22:00: Laboratório: Construção do Dashboard principal com KPIs do negócio.

Previsão de datas (podem mudar devido a feriados ou outras atividades)

Ciclo 3: Avançado, Integração e Qualidade

07/05 - Aula 13: Event-Driven e Realtime

18:40 - 22:00: Implementação de features reativos (websocket).

14/05 - Aula 14: Entrega 04 (Integração)

18:40 - 22:00: **ENTREGA 4:** Sistema integrando Domínio, IAM e Mensageria.

Demonstração de Teste de carga para upload de arquivos grandes (até 800MB).

21/05 - Aula 15: Testes de Software Corporativos

18:40 - 22:00: Escrita de testes para as regras de negócio mais críticas.

28/05 - Aula 16: Resiliência e Monitoramento

18:40 - 22:00: Implementação de **Health Checks** e **Circuit Breaker** e Configuração de logs de auditoria e monitoramento de erros.

Previsão de datas (podem mudar devido a feriados ou outras atividades)

Ciclo 4: Encerramento de projeto

04/06: *Período para ajuste de bugs e refinamento.*

11/06 - Aula 17: Refinamento da Documentação Técnica e Manual do Usuário.

18/06 - Aula 18: ENTREGA 5: Release Final (RC). Entrega de todo o código e documentação.

25/06 - Aula 19: Workshop de Deploy: Publicação e homologação final.

02/07 - Aula 20: DEFESA FINAL: Apresentação para banca com demonstração real.

Estruturação das apresentações:

- **Por dia, sem falta, 10-12 apresentações:**
- **Máximo 10-15 minutos (3 minutos para perguntas)**

Introdução: Nome do sistema e problema que resolve.

Demonstração Prática: Mostrar principais funcionalidades

Arquitetura e Tecnologias: Stack utilizada, decisões técnicas importantes.

Diferenciais Implementados: Funcionalidades extras e inovações.

Desafios e Soluções: Principais dificuldades enfrentadas.

Perguntas: Resposta às questões do professor/colegas.

Entrega 1

Definição do Projeto

Documento de Definição de Escopo

1. Identificação e Visão Estratégica

Esta seção situa o projeto no mercado e define o valor que ele entrega.

Nome e Descrição do Projeto: Identificação clara do sistema.

Justificativa e Objetivo: Por que o sistema está sendo construído e o que ele pretende resolver.

Stakeholders: Identificação dos atores envolvidos (Administradores, Clientes, Professores).

2. Escopo do Domínio

Aqui você insere a inteligência de domínio necessária para o 5º período.

Glossário da Linguagem Ubíqua: Tabela definindo os termos técnicos e de negócio para que não haja ambiguidade (ex: o que é um "Projeto" vs "Etapa").

Identificação de Bounded Contexts: Divisão lógica do sistema (ex: Contexto de Produção, Contexto de Identidade, Contexto de Notificação).

Entidades e Agregados Críticos: Descrição simplificada das grandes coleções de dados que devem ser consistentes (ex: Agregado de Projeto que engloba Tarefas e Arquivos).

Documento de Definição de Escopo

3. Escopo Funcional e Limites

Detalhamento técnico do que o sistema faz e, principalmente, do que ele **não** faz.

Funcionalidades Incluídas (RF): Lista de requisitos funcionais priorizados (Essencial, Importante, Desejável).

Funcionalidades Excluídas: O que não será desenvolvido (ex: edição de vídeo nativa, app mobile nativo).

Regras de Negócio (RN): As leis que regem o domínio (ex: "Um projeto só pode ser concluído se todas as etapas estiverem finalizadas").

Documento de Definição de Escopo

4. Arquitetura e Restrições Técnicas

Define as ferramentas e os atributos de qualidade do sistema corporativo.

Stack Tecnológica: Tecnologias previstas para Frontend, Backend e Banco de Dados.

Restrições: Limites de tempo (ex: 4 meses), linguagem (JavaScript/TypeScript) e infraestrutura.

Requisitos Não Funcionais (RNF): Atributos de qualidade como desempenho (2s por requisição), segurança (TLS/JWT) e disponibilidade (99%).

5. Riscos e Premissas

Preparação para imprevistos durante o semestre.

Premissas: Condições aceitas como verdadeiras (ex: acesso contínuo à internet, disponibilidade de APIs).

Riscos Iniciais: Mapeamento de possíveis falhas (ex: prazos apertados, dificuldades com upload de arquivos grandes).

O que deve conter no DDE

1. Glossário de Linguagem Ubíqua

2. Definição de Bounded Contexts (Contextos Delimitados)

Em vez de um monólito de requisitos, o escopo deve delimitar onde cada regra se aplica.

Fronteiras Lógicas: Identificar quais funcionalidades pertencem ao "Core Domain" (ex: Gestão de Produção) e quais são subdomínios de suporte (ex: Chat/Comunicação ou Suporte Técnico).

Separação de Responsabilidades: Deixar claro que um "Usuário" no contexto de Autenticação tem propriedades diferentes de um "Participante" no contexto de execução de tarefas.

3. Requisitos Funcionais por Contexto

Agrupar os requisitos funcionais (RF) de acordo com os contextos identificados para facilitar a modelagem táctica posterior.

Regras de Negócio (RN): Devem ser detalhadas como invariants do domínio (ex: "Um projeto só pode ser concluído quando todas as etapas estiverem finalizadas")

Priorização para o MVP: Classificar o que é "Essencial" para o funcionamento do domínio principal, deixando funcionalidades desejáveis para iterações futuras.

4. Escopo de Dados e Agregados

Identificar os grandes grupos de informações que precisam de consistência transaccional.

Identificação de Entidades Críticas: Listar o que possui identidade única (ex: Projetos, Tarefas) e o que pode ser tratado como valor (ex: Status, Prioridade).

Relacionamentos e Dependências: Esboçar como os arquivos e mensagens se vinculam obrigatoriamente a um projeto, definindo os limites de consistência.